

Runbook — Test banc : générateur de signal → SDR → lakehouse

Objectif : capturer un signal connu (générateur CW) avec un **RTL-SDR** ou un **BladeRF**, décrit par un **fichier de config JSON**, et le ranger dans le lakehouse MinIO au format SigMF.

Méthode rapide (un seul appel)

Une fois le matériel branché (voir §2) et ton JSON prêt (voir §3) :

```
cd ~/code/lassena/aerolake
./acquire.sh examples/test-rtlsdr.json      # ou ton propre fichier de config
```

`acquire.sh` enchaîne tout : pont SoapySDR → MinIO → healthcheck → capture. Tu n'as plus qu'à répondre **y** à « Push this capture to MinIO? ».

Les sections ci-dessous détaillent chaque étape (utile pour comprendre ou dépanner). Si `acquire.sh` marche, tu peux sauter directement à la §4 (vérif).

1. Prérequis logiciels (ce que fait `acquire.sh`)

```
cd ~/code/lassena/aerolake
bash setup-soapy.sh          # pont SoapySDR (À RELANCER après chaque `uv sync`)
cd docker && docker compose up -d && cd ..  # MinIO (le lakehouse)
uv run aerolake-healthcheck  # doit être vert
SoapySDRUtil --find          # doit lister ton SDR une fois branché
```

2. Montage matériel

1. **Câble coaxial** : sortie du générateur → entrée **RX** du SDR (SMA). - RTL-SDR : le port antenne. BladeRF : port **RX1** (ou **RX**).
 2. (!) **PUISSANCE — pour ne rien griller** : - Commence **TRÈS BAS** : **-40 dBm** (voire moins). - **Ne dépasse jamais ~-10 dBm** sur un RTL-SDR. - Tu **monteras** ensuite, petit à petit, jusqu'à voir la tonalité.
 3. **Fréquence** : RTL-SDR 24 MHz → 1.7 GHz · BladeRF 47 MHz → 6 GHz.
 4. (!) **Décalage (offset)** : règle le générateur **~250 kHz AU-DESSUS** du `center_freq` de la config. Le SDR a un pic parasite pile au centre (« DC spike ») ; en décalant, ta tonalité apparaît à **+250 kHz**, bien visible.
-

3. Le fichier de config JSON

Où le mettre ?

N'importe où — tu passes simplement son **chemin** en argument :

```
./acquire.sh chemin/vers/ma-config.json
```

Le plus simple : le ranger dans **exemples/** (à côté des modèles). Deux configs sont **déjà prêtes**, tu peux juste les éditer : - `exemples/test-rtlsdr.json` - `exemples/test-bladerf.json`

Ou crée le tien, ex. `exemples/mon-test.json`, puis `./acquire.sh exemples/mon-test.json`.

Comment le remplir ?

Minimal (le strict nécessaire pour un vrai SDR) :

```
{
  "signal_type": "test_banc",
  "center_freq": 100000000,
  "sample_rate": 2000000,
  "duration_s": 2.0,
  "source": { "type": "soapy", "driver": "rtlsdr", "agc": true }
}
```

Complet (un maximum de métadonnées) :

```
{
  "signal_type": "gnss_l1",
  "signal_type_detail": "L1 C/A",
  "center_freq": 1575420000,
  "sample_rate": 2000000,
  "duration_s": 5.0,

  "source": { "type": "soapy", "driver": "bladerf", "agc": false, "antenna": "RX1" },

  "author": "Theo Schmitt",
  "description": "Capture banc GNSS L1",
  "license": "https://creativecommons.org/licenses/by-sa/4.0/",
  "operator": "schmitt",

  "location": {
    "name": "labo LASSENA",
    "mobile": false,
    "geolocation": { "latitude": 45.4946, "longitude": -73.5623, "altitude": 50.0 }
  },

  "annotation": {
    "label": "porteuse",
    "comment": "tonalite du generateur",
    "freq_lower_edge": 1575320000,
    "freq_upper_edge": 1575520000
  },

  "antenna": { "model": "Tallysman TW3742", "type": "patch actif", "gain": 28.0 }
}
```

Les champs

Champ	Sens
-------	------

signal_type (requis)	identifiant court → dossier MiniIO + tag (gnss_l1, test_banc...)
center_freq (requis)	fréquence centrale en Hz (= fréquence générateur – 250 000)
sample_rate (requis)	échantillonnage en Hz (2000000 = 2 MS/s, OK RTL-SDR et BladeRF)
duration_s (requis)	durée en secondes (garde court : 2 s)
source.type	"soapy" (vrai SDR) ou "synthetic" (test sans matériel)
source.driver	rtlsdr ou bladerf
source.agc	true = gain auto · false = gain fixe
source.antenna	port (BladeRF RX1) — optionnel, à enlever si erreur
author / description / license / operator	métadonnées descriptives
location.name / mobile	lieu (devient un tag) · mobile ?
location.geolocation	latitude / longitude / altitude — ou "gps": true (lit gpsd)
annotation	label, comment, freq_lower_edge + freq_upper_edge (les deux ensemble)
antenna	model (requis si le bloc est présent), type, gain, polarization...

Règle stricte : un champ **inconnu = erreur** (anti-faute de frappe). Référence exhaustive : `examples/capture.full.json` et `examples/README.md`.

Exemple de pairage fréquence (RTL-SDR) : générateur à **100.25 MHz** → `center_freq: 100000000` (100.00 MHz) → la tonalité ressort à **+250 kHz**.

4. Lancer + vérifier

```
./acquire.sh examples/test-rtlsdr.json # capture, puis répondre "y" pour pousser
uv run aerolake-list --signal-type test_banc # la capture apparaît ?
```

(Les warnings Avahi / RtApi au démarrage = bruit SoapySDR, à ignorer.)

Voir le contenu (optionnel)

`aerolake-list` confirme que la capture **existe** ; pour regarder le **spectre** (et confirmer que ta tonalité est bien là, au bon niveau), ouvre le `.sigmf-data` dans **GNU Radio** (playback.grc) ou dans **Inspectrum**.

5. Dépannage

Symptôme	Cause / solution
No SDR found for driver=...	Appareil pas branché, mauvais driver, ou droits USB. Vérifie <code>SoapySDRUtil --find</code> . Au besoin règles udev ou <code>sudo</code> .
ModuleNotFoundError: SoapySDR	Relance <code>bash setup-soapy.sh</code> (le pont saute après un <code>uv sync</code>).
Un seul pic pile au centre (0 kHz)	C'est le DC spike ; décale le générateur de +250 kHz.
Niveau ~0 dBFS / écrêté	Baisse le niveau du générateur.
Pas de pic / niveau très bas	Monte le générateur ; vérifie câble + fréquence + offset.
MinIO injoignable	<code>cd docker && docker compose up -d</code> puis <code>uv run aerolake-healthcheck</code> .
BladeRF : erreur sur l'antenne	Enlève le champ "antenna" de la config.

Limite connue (gain)

La config n'expose pas le **gain** : c'est l'AGC si `"agc": true`, sinon un gain fixe de 40 dB. Pour un test au générateur, **le niveau du générateur est ton vrai bouton de réglage**.